

Ynara — Informe Técnico

Stack tecnológico y selección de modelos LLM

Equipo: 3 personas

Hardware disponible: RTX 4080 Super (16 GB VRAM)

Posicionamiento estratégico: infraestructura propia como ventaja defensiva frente a big tech (Google, OpenAI, Apple)

Target: estudiantes y profesionales 18-30, LATAM, arranque en Argentina con expansión a México, Colombia y Chile

Documento: Mayo 2026

CONTENIDO

- Modelos LLM — arquitectura dual ruteada e interacción Gemma + Qwen
- Stack tecnológico — Web, Mobile, Backend, Capa LLM, Base de datos, Infraestructura
- Curva de aprendizaje del equipo
- Tabla resumen de decisiones

1. Modelos LLM

1.1 Decisión: arquitectura dual ruteada

Ynara opera con **dos modelos open-source corriendo en paralelo**, con un router que decide cuál atiende cada mensaje según la naturaleza de la interacción. Esta arquitectura no es accidental: cada modelo gana en una dimensión donde el otro pierde, y combinarlos es lo que permite que Ynara sea simultáneamente cálida en lo conversacional y precisa en lo ejecutivo.

Las dos opciones que se consideraban como modelo único — Gemma 4 sola o Qwen 3.5 sola — implicaban ceder en una de las dos dimensiones críticas del producto. La decisión dual elimina ese trade-off.

1.2 Gemma 4 26B-A4B — el modelo conversacional

Quién es: modelo open-source de Google DeepMind, liberado en abril 2026, arquitectura MoE (Mixture of Experts) con 26B parámetros totales y solo 4B activos por token, lo que lo hace eficiente para inferencia local.

Licencia: Apache 2.0 (totalmente libre para uso comercial, sin atribución obligatoria, sin cláusulas de naming).

Para qué se usa en Ynara: los tres modos donde lo que importa es el registro humano del español rioplatense, la calidez emocional y el tono natural de acompañamiento.

- **Modo Bienestar** — el usuario habla de su estado emocional, descarga, busca ser escuchado. Gemma valida antes de aconsejar, no escala innecesariamente, no se vuelve clínica ni moralizadora. La generación anterior (Gemma 2-9b) fue el mejor modelo open-source en el único paper académico que evaluó variedades del español incluyendo rioplatense específicamente.
- **Modo Vida** — charla casual, recomendaciones, contextos cotidianos donde hace falta voseo natural, argentinismos sin forzar, y evitar deslices peninsulares.
- **Modo Estudio** — explicaciones, tutoría, procesamiento de textos. Aquí entra en juego el «thinking mode» nativo de Gemma 4, que permite razonamiento paso a paso manteniendo el tono de compañero, no de manual.

Justificación técnica: Gemma 4 fue entrenada en 140+ idiomas (vs los 12 de Llama 4), tiene la mayor amplitud multilingüe de la generación actual, y su predecesor lideraba el benchmark de inteligencia emocional EQ-Bench 3 entre los modelos open-source comparables. La arquitectura MoE permite que en una RTX 4080 Super corra cuantizada a 4 bits con throughput aceptable.

1.3 Qwen 3.5-9B — el modelo agente

Quién es: modelo open-source de Alibaba (Qwen Team), liberado en marzo 2026, arquitectura densa de 9B parámetros con Gated DeltaNet (atención lineal optimizada para coherencia en conversaciones largas).

Licencia: Apache 2.0.

Para qué se usa en Ynara: los dos modos donde lo que importa es ejecutar acciones, llamar herramientas correctamente, y mantener estado coherente a lo largo de muchos turnos.

- **Modo Productividad** — el usuario pide acciones concretas: «agendame mañana a las 10 con Lucas, recordame revisar el informe a la tarde». Qwen entiende la intención, decide qué herramienta llamar (`calendar.create_event`, `reminder.set`), arma el JSON correcto, lo ejecuta sin alucinar, y devuelve confirmación.
- **Modo Memoria** — el usuario pregunta sobre lo que se conversó hace días o semanas. Qwen rastrea el contexto guardado, recupera lo relevante, y responde de manera coherente sin perderse ni contradecirse.

Justificación técnica: Qwen 3.5-9B es objetivamente superior en tool calling (BFCL-V4 = 66.1, τ^2 -bench = 79.1) y mantiene mejor coherencia en conversaciones largas que Gemma o Llama de tamaño comparable. Su arquitectura Gated DeltaNet está diseñada específicamente para loops de agente sostenidos. Soporta 201 idiomas, contexto nativo de 262K tokens, y se integra de forma nativa con vLLM y Ollama mediante el parser `qwen3_coder` para tool calling.

1.4 Por qué Llama 4 quedó descartado

A pesar de ser la opción más mediática y tener el ecosistema más grande, Llama 4 se descartó por razones objetivas que no son negociables para Ynara:

- **Licencia contaminante:** la Llama Community License obliga a anteponer «Llama-» en el nombre de cualquier modelo derivado y a mostrar «Built with Llama» en el sitio. Para una marca que se llama Ynara y se posiciona en «infra propia», es un compromiso de marca innecesario.
- **Cobertura lingüística limitada:** soporta solo 12 idiomas oficialmente, contra 140+ de Gemma y 201 de Qwen. El español rioplatense específicamente es un caso donde su predecesor (Llama 3.1-8B) sacó 9.6/30, mientras Gemma 2-9b sacó 16.7/30 en el mismo benchmark.
- **Tamaño no apto para self-host:** la variante más chica (Llama 4 Scout) tiene 109B parámetros totales y requiere un H100 de 80 GB para correr cuantizada. No entra en una 4080 Super.
- **Calidez baja:** Llama 4 Scout tiene un score de 471.6 en EQ-Bench 3, contra 1043.4 de Gemma 3-27B. Diferencia significativa para un asistente que busca ser percibido como cálido.
- **Antecedente reputacional:** la controversia de LMArena en abril 2025, donde Meta envió una variante «experimental» diferente al modelo público para inflar el ranking, es un riesgo reputacional para una marca que vende honestidad técnica.

1.5 Cómo conviven los dos modelos en producción

El usuario nunca percibe que hay dos modelos. Hay un router al frente que decide cuál atiende cada mensaje. Para el usuario, todo es Ynara. Para Ynara, hay dos cerebros especializados.

1.5.1 El flow turn-por-turn

Cuando llega un mensaje del usuario, pasan tres cosas en orden:

1. El router clasifica la intención. Es una función chiquita en FastAPI (reglas + un clasificador rápido) que decide:

- ¿El mensaje pide ejecutar una acción concreta? (agendar, recordar, buscar en memoria, cambiar de modo) → **Qwen 3.5**
- ¿El mensaje es conversación, descarga, charla, pregunta abierta? → **Gemma 4**

2. Se arma el prompt con memoria inyectada. Antes de mandar el mensaje al modelo elegido, FastAPI hace una query a Postgres + Mem0 y trae:

- Top 3 hechos semánticos relevantes (perfil del usuario)
- Top 2 eventos episódicos recientes (qué pasó en los últimos turnos)
- El fragmento procedural activo (cómo hablarle a este usuario específico — voseo, registro, ritmo)

Eso se inyecta como contexto. **La memoria es la misma para ambos modelos** — el store es uno solo en Postgres. Cambia qué slice se inyecta según el modo activo, no la fuente.

3. El modelo elegido responde. El usuario recibe la respuesta. La conversación sigue.

1.5.2 Quién hace qué exactamente

Gemma 4 26B-A4B (modelo conversacional):

- Atiende los modos Bienestar, Vida, Estudio

- **Solo lee** memoria, no escribe
- No llama herramientas, no toca la agenda, no edita memoria
- Su trabajo es generar respuesta cálida, en rioplatense, con el tono procedural correcto para ese usuario

Qwen 3.5-9B (modelo agente):

- Atiende los modos Productividad y Memoria
- **Lee y escribe** memoria
- Llama herramientas: `calendar.create_event`, `reminder.set`, `memory.add`, `memory.search`, `memory.update`
- Su trabajo es ejecutar acciones, devolver JSON correcto, mantener coherencia en loops de varios turnos

1.5.3 Quién escribe en la memoria

Acá está la decisión arquitectónica clave. **Gemma 4 no escribe nunca en la memoria directamente**, pero la memoria igual se actualiza con cada conversación, incluso cuando responde Gemma.

¿Cómo? **Después** de cada turno (de cualquiera de los dos modelos), se dispara un job en Celery que toma el par (mensaje del usuario, respuesta del modelo) y le pide a **Qwen 3.5** que haga la extracción de hechos nuevos. Qwen es el que sabe devolver JSON limpio y estructurado, así que es el que escribe.

O sea: Gemma genera la conversación. Qwen, en background, lee la conversación de Gemma y decide qué guardar como memoria — siguiendo el patrón ADD/UPDATE/DELETE/NOOP de Mem0. Esto desacopla las responsabilidades: el modelo cálido se concentra en hablar bien, el modelo preciso se concentra en estructurar datos.

1.5.4 Cuándo se cambia de modelo a mitad de conversación

A veces el usuario empieza charlando (Gemma respondiendo) y de golpe pide algo concreto: «che, agendame esto». En ese momento el router detecta intención de acción y **el siguiente turno lo agarra Qwen**. La memoria es la misma, el contexto se pasa, el usuario no nota nada — solo que de golpe Ynara confirma «listo, agendado para mañana 10».

Después puede volver a charlar y vuelve Gemma. El cambio es invisible porque ambos modelos comparten el mismo contexto, la misma memoria, y el mismo fragmento procedural que les dice cómo hablarle a ese usuario.

1.5.5 La metáfora simple

Gemma es la cara y la voz. Qwen es las manos y la libreta. El router es el sistema nervioso que decide quién actúa según lo que el usuario pida. Ambos modelos se sirven desde el mismo servidor de inferencia (vLLM corriendo en la RTX 4080 Super), comparten el mismo store de memoria en Postgres, y para el usuario son una sola entidad: Ynara.

2. Stack tecnológico

2.1 Web — Next.js 15

Decisión: seguir con el stack que el equipo ya maneja, ampliándolo con las herramientas modernas del ecosistema.

Tecnologías:

- Next.js 15 con App Router + TypeScript
- Tailwind CSS + shadcn/ui (componentes accesibles)
- GSAP + Lenis (animaciones — lo que ya manejan)
- TanStack Query (fetching y cache)
- Zustand (state management liviano)
- React Hook Form + Zod (validación tipada)
- Auth.js v5 (autenticación)
- Vercel (deploy free tier durante MVP)

Por qué: las tres personas del equipo ya manejan este stack. Cero curva de aprendizaje en la pieza más visible del producto (landing, dashboard, panel admin). Vercel free tier alcanza para todo el MVP. El stack es el estándar moderno de Next.js 2026, con buena documentación y comunidad enorme.

2.2 Mobile — Expo (React Native)

Decisión: Expo con React Native, descartando Flutter y desarrollo nativo.

Tecnologías:

- Expo SDK 52+ con TypeScript
- Expo Router (file-based routing, mismo modelo mental que Next.js App Router)
- NativeWind (Tailwind CSS para React Native)
- TanStack Query (compartido con web)
- Zustand (compartido con web)
- expo-secure-store (tokens de auth)
- expo-notifications (push para recordatorios — crítico para Ynara)
- EAS Build (compilación en la nube, free tier suficiente)

Por qué se elige Expo: TypeScript, JSX, hooks — la sintaxis y mental model que el equipo ya tiene de Next.js transfiere directamente. La gran ganancia es que entre 60% y 70% del código (lógica de negocio, validaciones, llamadas al backend, tipos compartidos) se reusa entre web y mobile. Para tres personas manteniendo iOS, Android y web, esto es la diferencia entre llegar a la defensa de tesis o no.

Por qué se descarta Flutter: obliga a aprender Dart, un lenguaje y ecosistema completamente paralelo. Tres personas que ya saben TypeScript no tienen razón para sumar Dart al stack.

Por qué se descarta nativo (Swift + Kotlin): mantener dos codebases nativos con tres personas es inviable en un horizonte de tesis. Aunque la performance es superior, el costo de mantenimiento mata el proyecto antes de que se valide el producto.

2.3 Backend — FastAPI (Python)

Decisión: FastAPI en Python, no Hono.js o Express en TypeScript.

Tecnologías:

- FastAPI + Pydantic v2 (validación de schemas tipada, equivalente a Zod en TS)
- SQLAlchemy 2.0 + Alembic (ORM y migraciones)
- Uvicorn + Gunicorn (servidor ASGI para producción)
- Celery + Redis (tareas en background: procesamiento de memoria, recaps periódicos, etc.)
- Authlib o fastapi-users (autenticación con JWT)
- uv (package manager Python moderno, mucho más rápido que pip)
- Ruff (linter)

Por qué Python y no TypeScript: el código de inferencia, fine-tuning del LoRA, manipulación de embeddings y orquestación del agente **es Python sí o sí**. No hay alternativa madura en TypeScript. Tener el backend completo en Python (en lugar de partir entre Node y Python) significa un solo lenguaje, un solo deploy, un solo set de dependencias. Reduce la fricción operativa de manera significativa.

Por qué FastAPI específicamente: es el estándar moderno de APIs en Python. Async-native (alto throughput), tipado con Pydantic (validación automática), documentación OpenAPI generada automáticamente, y la curva de aprendizaje viniendo de TypeScript es de aproximadamente una semana — los decoradores, los types y la validación de schemas son conceptos que ya manejan.

2.4 Capa LLM — vLLM + Unsloth + LlamaIndex

Decisión: stack especializado en Python para servir y entrenar los modelos.

Tecnologías:

- **vLLM** — servidor de inferencia, soporta tool calling de Qwen y Gemma de forma nativa, optimizado para throughput. Corre directamente sobre la 4080 Super.
- **Ollama** — alternativa más simple para desarrollo local (un comando y arranca).
- **Unsloth** — fine-tuning con QLoRA, 2x más rápido y 30% menos VRAM que TRL puro.
- **Hugging Face Transformers + PEFT** — para todo lo que Unsloth no cubra.
- **LlamaIndex** — orquestación del agente y sistema de memoria contextual.

Por qué LlamaIndex y no LangChain: LangChain es más conocido pero LlamaIndex está más enfocado en el problema de memoria contextual y RAG, que es el corazón de Ynara. Para un asistente donde la memoria es protagonista, LlamaIndex es la elección correcta.

2.5 Base de datos — PostgreSQL + pgvector

Decisión: un solo Postgres con la extensión pgvector, sin vector store separado.

Tecnologías:

- PostgreSQL 16 (datos principales: usuarios, conversaciones, eventos, recordatorios).
- pgvector (extensión nativa para embeddings y búsqueda semántica de la memoria contextual).
- Redis (cache, sesiones, cola de Celery).

Por qué no usar Pinecone, Weaviate o Qdrant separados: complejidad innecesaria para el MVP. pgvector dentro del mismo Postgres es suficiente hasta llegar a millones de embeddings, simplifica el stack drásticamente, y la migración a un vector store dedicado más adelante es trivial si la escala lo justifica. Es el camino que toman proyectos comparables como RigoChat-2 y la mayoría de las tesis serias del último año.

Sobre Supabase: durante el MVP es viable usar Supabase como Postgres gestionado, manteniendo la regla de no usar el cliente de Supabase desde el frontend (todo pasa por la API de FastAPI). Eso garantiza que la migración a Postgres self-hosted más adelante sea simplemente cambiar el connection string. La decisión final entre Supabase MVP o Postgres self-hosted desde día 1 depende de cuánto tiempo de DevOps quiera invertir el equipo en la primera semana.

2.6 Infraestructura

Decisión: la 4080 Super del equipo como GPU de inferencia y fine-tuning, complementada con servicios free tier para web y storage.

Componentes:

- **Web (Next.js)** — Vercel free tier.
- **Backend Python** — Docker + docker-compose, deploy en VPS modesto (Hetzner CX22, ~€4/mes) o en la propia PC con la 4080 expuesta vía Cloudflare Tunnel.
- **GPU inference** — RTX 4080 Super con vLLM corriendo en Linux (mejor performance que Windows/WSL).
- **Storage** — Cloudflare R2 (compatible S3, sin egress fees, free tier generoso).
- **Exposición de servicios** — Cloudflare Tunnel para servir la PC del equipo a internet sin abrir puertos, sin IP estática, gratis.
- **Monitoring** — Sentry (error tracking, free tier alcanza).
- **Analytics** — PostHog o Plausible (privacy-first).

Por qué este setup específicamente: cumple con el posicionamiento de «infra propia» del pitch de tesis sin costos durante el MVP. La 4080 Super tiene VRAM suficiente para correr Gemma 4 26B-A4B y Qwen 3.5-9B

cuantizadas a 4 bits, con throughput aceptable para servir múltiples usuarios concurrentes en escala MVP. Cloudflare Tunnel resuelve el problema de exposición a internet sin necesidad de IP estática ni configuración de redes complicada.

2.7 Tooling transversal

- **pnpm** o **bun** — package manager para JS/TS
- **Turborepo** — monorepo para compartir código entre web, mobile y paquete de tipos compartidos
- **Biome** o **ESLint + Prettier** — linting y formateo
- **GitHub Actions** — CI/CD (free para repos privados)
- **Sentry** — error tracking
- **PostHog** — analytics privacy-first

3. Curva de aprendizaje realista

Tres personas, viniendo del stack actual, necesitan aprender lo siguiente:

- **FastAPI + Pydantic + SQLAlchemy** — aproximadamente 1 semana para algo productivo, viniendo de TypeScript.
- **Expo / React Native** — aproximadamente 1 semana, casi todo lo de Next.js transfiere.
- **vLLM + Unsloth** — aproximadamente 1-2 semanas para tener el ciclo completo de fine-tune + serve funcionando.
- **Linux + Docker básico** — si no se maneja ya, sumar 3-5 días.

Total realista: 2-4 semanas de aprendizaje en paralelo al desarrollo, distribuidas entre las 3 personas. Es un costo asumible para una tesis y queda absorbido en el cronograma del MVP.

4. Tabla resumen de decisiones

Área	Decisión	Alternativas descartadas
Modelo conversacional	Gemma 4 26B-A4B	Qwen 3.5 sola, Llama 4
Modelo agente	Qwen 3.5-9B	Gemma 4 sola
Web	Next.js 15	Mantener stack actual ampliado
Mobile	Expo (React Native)	Flutter, nativo (Swift + Kotlin)

Área	Decisión	Alternativas descartadas
Backend	FastAPI (Python)	Hono.js, Express, Go
ORM	SQLAlchemy 2.0	Drizzle, Prisma
Base de datos	PostgreSQL + pgvector	Pinecone, Weaviate, Qdrant separados
Cache	Redis	Memcached
Inferencia LLM	vLLM	Ollama (solo dev local), Text Generation Inference
Fine-tuning	Unsloth + QLoRA	TRL puro, Axolotl
Orquestación agente	LlamaIndex	LangChain
GPU	RTX 4080 Super (propia)	RunPod, Vast.ai
Storage	Cloudflare R2	AWS S3, Backblaze B2
Deploy web	Vercel	Netlify, Cloudflare Pages
Exposición backend	Cloudflare Tunnel	ngrok, IP estática